

32-bit RISC Processor Design and Verification

1. Introduction

A processor is a digital system that executes instructions and performs computational operations. This project implements a simple 32-bit educational RISC processor using SystemVerilog. The processor uses a custom instruction encoding inspired by the MIPS instruction format for educational purposes and is not a complete RISC-V implementation.

2. Objectives

- Design a basic 32-bit RISC processor.
- Implement processor components using SystemVerilog.
- Understand the interaction between CPU datapath components.
- Implement instruction fetching and decoding.
- Implement register-based arithmetic operations.
- Develop individual testbenches and an integrated processor testbench.
- Verify functionality using simulation and GTKWave.
- Maintain the project using Git and GitHub.

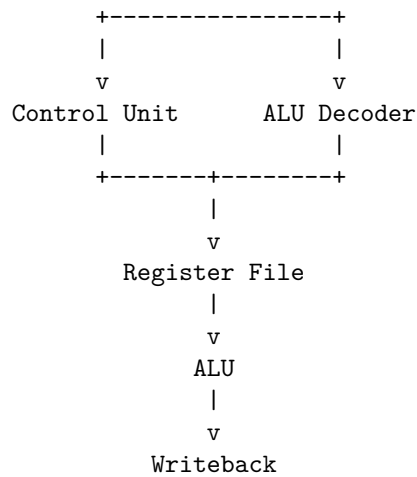
3. Tools and Technologies

Tool	Purpose
SystemVerilog	RTL design and verification
Icarus Verilog	HDL compilation and simulation
GTKWave	Waveform analysis
Visual Studio Code	Development
Ubuntu WSL	Linux development environment
Git	Version control
GitHub	Source repository

4. Processor Architecture

The processor consists of a Program Counter, Instruction Memory, Register File, Control Unit, ALU Decoder, and Arithmetic Logic Unit.

```
Program Counter
|
v
Instruction Memory
|
v
Instruction Decode
|
```



5. RTL Module Design

Program Counter

The Program Counter stores the address of the current instruction and increments by 4 bytes after each clock cycle. Reset initializes it to zero.

Instruction Memory

Instruction Memory contains 16 locations, each storing one 32-bit instruction.

Register File

The processor contains 32 registers of 32 bits each, with two read ports and one write port. R0 always reads as zero.

Arithmetic Logic Unit

The ALU supports ADD, SUB, AND, OR, XOR, and LESS THAN operations and generates a zero flag.

ALU Decoder

The ALU Decoder converts ALU operation information and the instruction function field into the ALU control signal.

Control Unit

The Control Unit decodes the opcode and provides control definitions for R-type, ADDI, LW, SW, and BEQ. The currently tested datapath fully executes ADDI and R-type ADD.

6. Instruction Set

The processor uses a custom 32-bit instruction format inspired by MIPS.

R-Type

| opcode | rs | rt | rd | shamt | funct |

I-Type

| opcode | rs | rt | immediate |

Test instructions: - ADDI R1, R0, 10 — 0x2001000A - ADDI R2, R0, 20 — 0x20020014 - ADD R3, R1, R2 — 0x00221820

7. Simulation Methodology

Each processor component was independently tested before integration. The complete processor was compiled with Icarus Verilog and simulated with a SystemVerilog testbench. A VCD waveform was generated for GTKWave analysis.

Compilation:

```
iverilog -g2012 -o risc_processor_sim rtl/program_counter.sv rtl/instruction_memory.sv rtl/
```

Execution:

```
vvp risc_processor_sim
```

8. Simulation Results

Time	PC	Instruction	ALU Result
5000	0x00000000	0x2001000A	0x0000000A
15000	0x00000004	0x20020014	0x00000014
25000	0x00000008	0x00221820	0x0000001E

0x0000001E is decimal 30, confirming $10 + 20 = 30$.

9. Functional Verification

The final processor testbench automatically checked the register file:

Register	Expected	Actual	Result
R1	10	10	PASS
R2	20	20	PASS
R3	30	30	PASS

Console verification:

PASS: R1 = 10

PASS: R2 = 20

PASS: R3 = 30

10. Waveform Analysis

The simulation generates `waveforms/risc_processor.vcd`. Important signals include `clk`, `reset`, `uut.pc`, `uut.instruction`, and `uut.alu_result`. The waveform demonstrates the relationship between clock cycles, instruction fetching, PC progression, and ALU results.

11. Verification of Individual Modules

Dedicated testbenches were developed for the Program Counter, Instruction Memory, Register File, ALU, ALU Decoder, Control Unit, and complete processor.

12. Current Limitations

The design is an educational processor rather than a complete commercial processor or complete RISC-V implementation. Data memory, complete LW/SW execution, BEQ branch PC selection, a larger instruction set, pipelining, hazard detection, and forwarding are not yet implemented.

13. Future Improvements

1. Implement data memory.
2. Implement LW and SW.
3. Implement BEQ branch execution.
4. Add jump and additional ALU instructions.
5. Expand instruction memory.
6. Improve automated verification.
7. Add pipelining.
8. Add hazard detection and forwarding.
9. Perform RTL synthesis.
10. Target an FPGA implementation.

14. Conclusion

This project successfully demonstrates the design and functional verification of a basic 32-bit educational RISC processor using SystemVerilog. It covers datapath design, instruction decoding, register operations, ALU control, simulation, waveform analysis, and automated verification.

Final verified result: R1 = 10, R2 = 20, R3 = 30.

Final Verification Status: PASS